“Money Manager: Full-Stack Financial Management System”

A MAJOR PROJECT

Submitted in Partial Fulfillment of the Requirement for the Award of the Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

SUBMITTED TO

Deenbandhu Chhotu Ram University of Science and Technology Murthal, Sonipat, Haryana

SUBMITTED BY

Manish Kumar (22015001025)

Mohammad Kashim (22015001030)

International Institute of Technology & Management, Murthal, Haryana, India

Department of Computer Science & Engineering International Institute of Technology & Management 49th Milestone, G.T. Karnal Road (NH-44), Murthal, Haryana-131039

CANDIDATE'S DECLARATION

We, Manish Kumar, Mohammad Kashim, student of Bachelor of Technology, Computer Science & Engineering, International Institute of Technology and Management, hereby declare that the work presented in the project entitled "Money Manager: Full-Stack Financial Management System" is the outcome of my own bona-fide work, which is correct to the best of my knowledge and this work has been carried out taking care of Engineering Ethics.

The work presented does not infringe any previous work and has not been submitted to any University for the award of any degree.

The project entitled "Money Manager: Full-Stack Financial Management System" being submitted has been examined by us and is hereby approved for the award of the degree "Bachelor of Technology in Computer Science & Engineering", for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.

Manish	Mohammad Kashim
(22015001025)	(22015001030)

CERTIFICATE

This is to certify that Manish Kumar, student of the Department of Computer Science and Engineering, has successfully completed his major project titled "Money Manager: Full-Stack Financial Management System" as part of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.
The project was undertaken under the guidance of **Mr.Sourav kumar.**

Signature of supervisor

Mr. Sourav Kumar

(IITM,Murthal)

Signature of external examiner

Acknowledgment

We would like to express our sincere gratitude to all those who have helped and supported us throughout the completion of our project titled "Money Manager: Full-Stack Financial Management System".

First and foremost, We would like to extend my heartfelt thanks to my project supervisor, **Mr.Sourav Kumar** , Department of Computer Science and Engineering, International Institute of Technology and Management, Murthal, Haryana, for providing me with invaluable guidance, support, and encouragement during the course of this project. Their expertise and insights were instrumental in the successful completion of my work.

We would also like to thank the faculty members and staff of the Department of Computer Science and Engineering for their constant support and for providing us with the resources and knowledge required to carry out this project.

Our sincere gratitude goes to the college, International Institute of Technology and Management, for creating an environment that promotes academic growth and innovation.

Lastly, We would like to thank our family and friends for their constant encouragement and understanding throughout the project. We am deeply grateful to everyone who directly or indirectly contributed to the success of this project.

Abstract

The Money Manager is a high-performance, full-stack web application designed to automate and simplify personal financial tracking. It allows users to efficiently log daily income and expenses, categorize transactions, and visualize spending habits through interactive dashboards. By leveraging a modern technology stack—React.js for the frontend, Spring Boot for the backend, PostgreSQL for secure data persistence, and Redis for high-speed caching—the system ensures low-latency data retrieval and robust security. The system also includes automated email reminders to help users stay within budget limits. By reducing manual ledger work and increasing data accessibility, this system aims to enhance overall financial literacy and management for its users.

INDEX

1 INTRODUCTION

- Introduction to the project
- Objective of the project
- Scope of the system
- Previous work or related systems

2 SYSTEM REQUIREMENTS

- Functional Requirements
- Non-Functional Requirements
- Hardware and Software Requirements

3 SYSTEM DESIGN

- Frontend Design (React.js UI/UX)
- Database Design (PostgreSQL)
- Relationships and Keys

4 IMPLEMENTATION AND CODING

- Explanation of the technologies used (Spring Boot, React, Redis, Postgres)
- Frontend Development
- Backend Development

5 TESTING

- Types of testing conducted
- Test cases and results

6 CONCLUSION

- Summary of the project
- Achievements and outcomes
- Future improvements or extensions

7 REFERENCES

- Websites, and other resources used

Chapter 1

INTRODUCTION

A Full-Stack Money Manager is a digital platform designed to manage personal or business financial operations efficiently. It allows users to log transactions, monitor cash flow, and access comprehensive financial analytics from an online interface. The system includes features for users to track spending by category, check current balances, and receive automated reminders.

Objective of the project:

The primary objective of the Money Manager project is to create an efficient and user-friendly platform that streamlines financial tracking. The key objectives include:

- **Easy Access:** Allow users to securely log in and manage their finances from any device.
- **Transaction Management:** Help users organize income and expenses systematically.
- **Performance Optimization:** Utilize Redis caching to ensure financial dashboards load instantly.
- **Automated Alerts:** Send system-generated email reminders for recurring bills and budget limits.

• **24/7 Availability**: Make the library accessible at any time, even when it's closed.

Scope of the system:

The scope includes allowing users to register, log in securely via JWT, and manage their profiles. Users can add, update, or delete financial transactions, and categorize them appropriately. The backend processes this data to generate real-time visual reports on the frontend. Additionally, the system manages scheduled background tasks for email notifications and utilizes in-memory caching to reduce database load for frequently requested data.

Previous work or related systems:

- **Manual Tracking:** Older methods required users to manually record expenses in physical ledgers or basic spreadsheets, which were prone to human error.

- **Offline Desktop Apps:** Earlier software required installation on a single machine, limiting accessibility and lacking automated cloud backups.

- Modern online financial systems, like this project, provide a more efficient solution by allowing remote access, instant analytics, automatic reminders, and scalable data storage.

Chapter 2

SYSTEM ANALYSIS

Functional Requirements:

- **User Registration and Login:** Secure authentication using Spring Security and JWT tokens.
- **Transaction Logging:** Users must be able to create, read, update, and delete (CRUD) their income and expense records.
- **Categorization:** The system must allow users to assign categories (e.g., Food, Rent, Salary) to transactions.
- **Dashboard Analytics:** The system must aggregate data to display total balances and monthly summaries.
- **Email Reminders:** Automated triggers for specific user conditions or scheduled intervals.

Non Functional requirements:

- **Performance**:

Fast response times for data retrieval, achieved via Redis caching.

- **Scalability**:

Microservices-ready backend design to accommodate growing user data.

- **Security**:

Password hashing and encrypted database connections to protect sensitive financial records.

- **Usability**:

  The system should have a user-friendly interface that is easy for both users and staff , admin to navigate.

- **Availability**:

  The system should be available 24/7 with minimal downtime, ensuring that users can access it anytime.

- **Reliability**:

  The system should be dependable, with minimal bugs or crashes, to ensure consistent performance and accuracy in handling tasks.

- **Maintainability**:

  The system should be easy to update and maintain, allowing for bug fixes, upgrades, or adding new features without major disruptions.

- **Backup and Recovery**:

There should be a backup system in place to protect data in case of failure and a way to recover data if the system goes down.

➢ **Compatibility**:
The system should work across different devices and web browsers, ensuring all users have access regardless of the platform they use.

These non-functional requirements help ensure the system runs efficiently, is secure, and provides a good user experience.

Hardware Requirements:

➢ **Server**:

- Processor: Intel i5 or higher
- RAM: At least 4 GB (8 GB is better)
- Storage: 500 GB SSD (or more)
- Stable internet connection

➢ **Client (User Devices)**:

- Desktop/Laptop with at least 4 GB RAM
- Mobile devices with internet access

Software Requirements:

➢ **Operating System**:

- **Development**: Windows, macOS, or Linux

➢ **Front End Development**:

- **React.js**: For creating the webpage structure
- **HTML5:**
- **Tailwind CSS**:
- **JavaScript**:

➢ **Back End Development**:

- **Java (Spring Boot)**: Spring Web, Spring Data JPA, Spring Security

➢ **Database**:

- **PostgreSQL**

➢ Caching:

- **Redis Server**:

➢ Development Tools:

- **IntelliJ IDEA**: for Spring boot development
- **Visual Studio Code :For React.js**
- **VS Code**
- **Git**
- **Postman: For API Testing**

Chapter 3

SYSTEM DESIGN

➢ **Frontend Design:**

Page layout: The **Page Layout** section will describe the structure, organization, and interaction of the main pages of the system. These layouts will serve as blueprints for the user interface (UI) design and the flow of interactions across different user roles (like Admin and User). It contains:

- **Navigation:** A simple navigation bar with links to the Home page, Login page, Admin Panel, and User Dashboard.

- **Home Page:** A welcome message indicating that it is the main page for all users.

- **Login Page:** A login form with fields for username and password and a login button. This form is likely intended for authentication purposes, allowing users to access their respective dashboards based on their credentials.

- **Admin Panel:** A brief description suggesting that this section is for managing users and settings. This implies that administrators would use this panel to control various aspects of the library system, such as adding/removing users, managing books, and configuring system settings.

- **User Dashboard:** A welcome message to the user's personal dashboard. This suggests that users, once logged in, would be able

to access their personalized information and perform actions specific to their roles, such as borrowing books, checking their account details, or viewing their reading history.

Basic Structure:

1. Header

The **header** is the top part of a webpage. It usually contains:

- **Logo**: A picture or text that represents the website.
- **Navigation Links**: Links to important pages like Home, Search, Login, etc.
- **Search Bar**: A field where users can type to search for books or content.

2. Footer

The **footer** is the bottom part of the webpage. It typically includes:

- **Additional Links**: Links to pages like Privacy Policy, Terms of Service, and Contact Us.
- **Social Media Links**: Icons for platforms like Facebook or Twitter. • **Copyright Information**: A note about who owns the website or content.

3. Forms

Forms are sections where users can input their information. Common forms include:

- **Login Form**: Where users enter their username/email and password.
- **Registration Form**: Where new users create an account by entering their details.
- **Search Form**: Where users search for books by typing keywords.

Frontend Design:

<CODING PART>

➢ Database Design

In the development of an **Online Library Management System (OLMS)** based on the **Software Development Life Cycle (SDLC)**, creating well-structured tables for users, books, and transactions is essential for maintaining an organized database that ensures efficient operations. These tables hold critical data for the system's functionality, and each of them serves a specific purpose.

1. Users Table

This table stores details about the people who use the library. Each user (member or librarian) has a unique ID and information like their name, email, and role (e.g., member, librarian).

- **Important fields**: `user_id`, `first_name`, `last_name`, `email`, `role`, `membership_date`
- **Purpose**: Tracks who is using the library.

2. Books Table

This table stores information about all the books in the library, including the title, author, ISBN number, and how many copies are available.

- **Important fields**: `book_id`, `title`, `author`, `ISBN`, `copies_available`, `genre`
- **Purpose**: Keeps track of all books and their availability.

3. Transactions Table

This table keeps track of the borrowing and returning of books. It records which user borrowed which book, when they borrowed it,

when it's due to be returned.

- **Important fields**: `transaction_id`, `user_id` (links to the Users table), `book_id` (links to the Books table), `transaction_date`, `due_date`, `return_date`, `status`
- **Purpose**: Manages the books being borrowed and returned, along with tracking due dates and fines.

➢ **Relationships: Users and Books(Borrowing, Transactions)**

In your **Online Library Management System**, the relationship between **Users** and **Books** (through **Borrowing** and **Transactions**) works like this:

1. Users:

- **Users** are the people who register on your system to borrow books. • Each user can borrow multiple books.

2. Books:

- **Books** are the items in the library that users can borrow. • Each book can be borrowed by multiple users, but at different times.

3. Borrowing:

- When a user wants to borrow a book, a **borrowing record** is created.
- This record stores details like:
  - Which user borrowed the book
  - When the book was borrowed
  - When it is due back
  - Whether the book is returned or overdue

4. Transactions:

- A **transaction** tracks every action related to borrowing a book. This includes:
  - Borrowing the book (when the user checks it out)
  - Returning the book (when the user brings it back)
  - Any fines if the book is returned late

Relationships:

- **User to Borrowing**: A user can have multiple borrowing records (one for each book borrowed).
- **Book to Borrowing**: A book can be borrowed multiple times, but at different times by different users.
- **User to Transaction**: A user can have multiple transactions (borrowing, returning, fines).
- **Book to Transaction**: A book can be involved in multiple transactions (borrowed, returned, fined).

In short, **Users** borrow **Books**, and each borrowing is tracked through **Transactions**. These records help the system manage who has which books and when they need to be returned.

➢ **Primary key and Foreign key**

In database design, primary keys and foreign keys are used to establish relationships between different tables and ensure data integrity. Here's an explanation in simple terms:

1. Primary Key:

- A Primary Key is a unique identifier for each record in a table. • It ensures that each record can be uniquely identified. • No two records in a table can have the same primary key value. • A primary key cannot be null (empty), meaning it must have a value for every record.

2. Foreign Key:

- A Foreign Key is a field (or a combination of fields) in one table that refers to the Primary Key in another table.
- It creates a link between two tables and enforces relationships between them.
- A foreign key allows you to combine information from multiple tables in your queries.

In short we can say that Primary Key is a unique identifier for records in a table. Foreign Key is a reference to the primary key in another table, creating a relationship between the two tables.

Chapter 4

IMPLEMENTATION & CODING

➢ Technologies Used

HTML (HyperText Markup Language)

Think of **HTML** as the skeleton of your website. It provides the structure and layout of your web pages. HTML tells the browser what content to display and how to organize it. When you create a webpage, you're using HTML to define different elements like headings, paragraphs, buttons, and forms.

For example, when a user visits your online library system, HTML creates:

- A **login form** where users can enter their details.
- A **list of books** showing their titles, authors, and availability.
- A **button** that users can click to borrow or return a book.

CSS (Cascading Style Sheets)

Now, after HTML provides the structure of your web pages, CSS is what makes everything look good. It controls the style of the HTML elements, like colors, fonts, spacing, and layout.

For example:

- CSS can make the background of your homepage a light color.
- It can change the font of book titles to make them bold and large.
- It can also organize the books in neat rows or columns on the page.
- It ensures the buttons look clickable, with colors changing when users hover over them.

CSS allows you to customize the visual appearance of your online library system so that users have a pleasant, easy-to-navigate experience.

SQL (Structured Query Language)

SQL is used to manage and interact with the database of your library system. While HTML builds the structure of the site and CSS makes it visually appealing, SQL helps store and retrieve all the important information.

In an online library system, your database might store:

- **Books** (titles, authors, availability)
- **Users** (names, emails, borrowed books)
- **Transactions** (books borrowed, due dates, return dates)

SQL allows you to:

- **Add new books** to the system when new stock arrives. • **Update the number of available copies** of a book when someone borrows or returns it.
- **Check which books are currently borrowed** by a specific user. • **Track overdue books** and apply fines if needed.

When a user wants to borrow a book, SQL will check if it's available. When they return it, SQL updates the records to show the book is back in stock.

➢ Frontend Development

Creating HTML Pages and CSS Styling (Home, Login, Admin Panel):

HTML Code For Home page:

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial scale=1.0">
 <title>Library Management System</title>
 <link rel="stylesheet" href="styles.css">
</head>
<body>
 <div class="container">
 <h1>Welcome to the Library Management System</h1> <p>Your one-stop solution for managing books and users.</p> <div class="buttons">
 <a href="login.html" class="btn">Login</a> <a href="register.html" class="btn">Register</a> </div>
 </div>
</body>
```

```
</html>
```

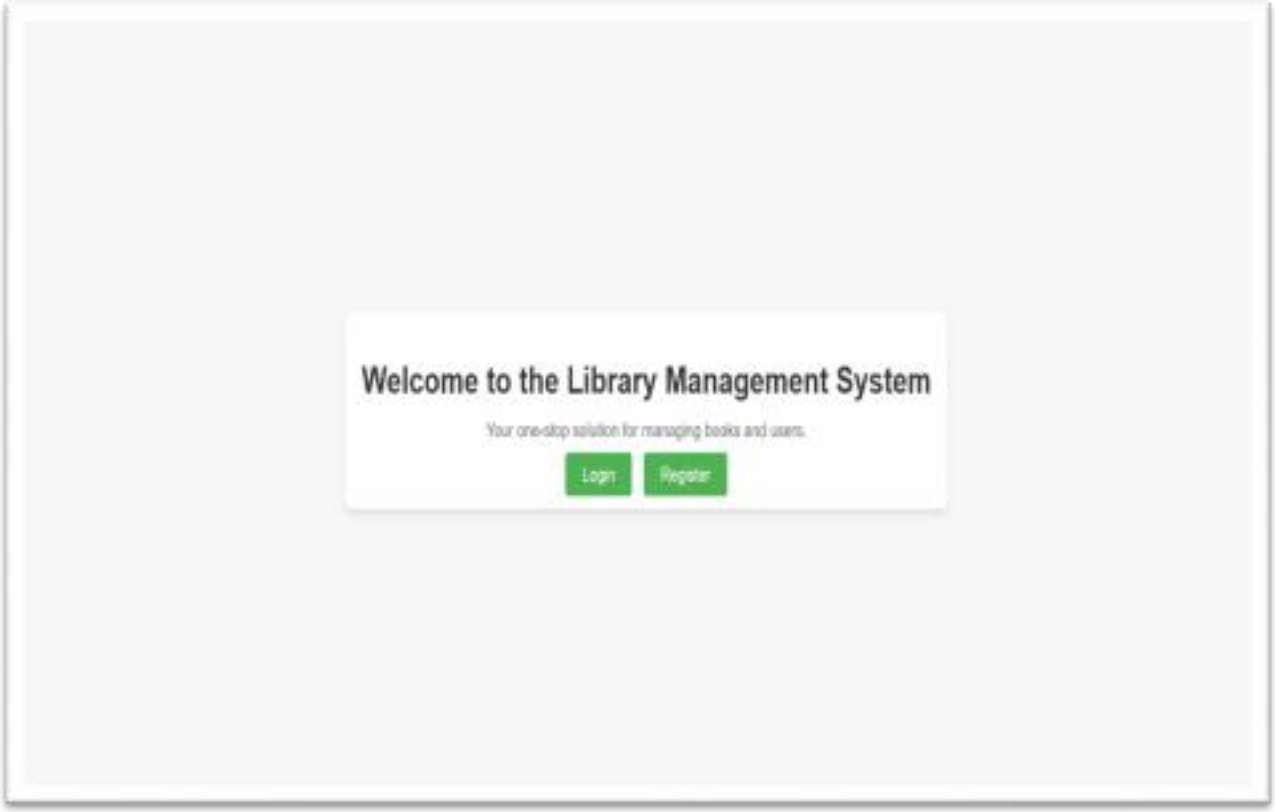
CSS Style For Home Page:

```
body {
 font-family: Arial, sans-serif;
 background-color:    #f5f5f5;
 margin: 0;
 padding:     0;
 display: flex;
 justify-content: center;
 align-items:      center;
 height: 100vh;
 }
.container {
 text-align:          center;
 background-color: #fff;
 padding: 20px;
 border-radius: 8px;
 box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1); }

h1 {
 color: #333;
 margin-bottom: 10px;
 }
p {
 color: #666;
 margin-bottom: 20px;
 }
.buttons .btn {
 text-decoration: none;
```

```
background-color:        #4CAF50;
color: #fff;
padding: 10px 20px;
border-radius:   4px;
margin: 5px;
}
.buttons       .btn:hover       {
background-color: #45a049;
}
```

Output of Home Page:



HTML Code For Login Page:

```
<!DOCTYPE html>

<html lang="en">

<head>

 <meta charset="UTF-8">

 <meta name="viewport" content="width=device-width, initial scale=1.0">

 <title>Login - Library Management</title>

 <link rel="stylesheet" href="style.css">

</head>

<body>

 <div class="form-container">

 <h2>Login</h2>

 <form action="dashboard.html" method="post">

<label for="email">Email:</label>

 <input type="email" id="email" name="email" placeholder="Enter your email" required>

 <label for="password">Password:</label>

 <input type="password" id="password" name="password" placeholder="Enter your password" required>

 <button type="submit">Login</button>

 </form>

 <p>Don't have an account? <a href="register.html">Register
```

```
here</a>.</p>

 </div>

</body>
</html>
```

CSS Style for login page:

```
.form-container          {
 background-color: #fff;
 padding: 20px;
 border-radius: 8px;
 box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
width: 400px;
 margin: 0 auto;
 margin-top: 10%;
}
h2 {
 text-align: center;
 color: #333;
}

label {
 display:      block;
 margin-top: 15px;
 color: #555;
}

input {
 width:      100%;
 padding:    10px;
 margin-top: 5px;
 border: 1px solid #ccc;
 border-radius: 4px;
 box-sizing: border-box;
}
```

```
button          {
 width: 100%;
 padding:      10px;
 margin-top: 20px;
 background-color: #4CAF50;
 color: white;
 border:          none;
 border-radius:  4px;
 cursor: pointer;
}
button:hover {
 background-color: #45a049;
}

p {
 text-align:  center;
 margin-top: 20px;
 color: #666;
}

p a {
 color: #4CAF50;
 text-decoration: none;
}

p a:hover {
 text-decoration: underline;
}
```

Output for Login Page:

HTML Code For Admin Panel:

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial scale=1.0">
 <title>Admin Panel</title>
 <link rel="stylesheet" href="styless.css">
</head>
<body>
 <div class="container">
 <h1>Admin Panel</h1>
 <div class="admin-actions">
 <a href="addBook.html" class="btn">Add Book</a> <a href="viewBooks.html" class="btn">View Books</a> <a href="manageUsers.html" class="btn">Manage Users</a> </div>
 </div>
</body>
```

</html>

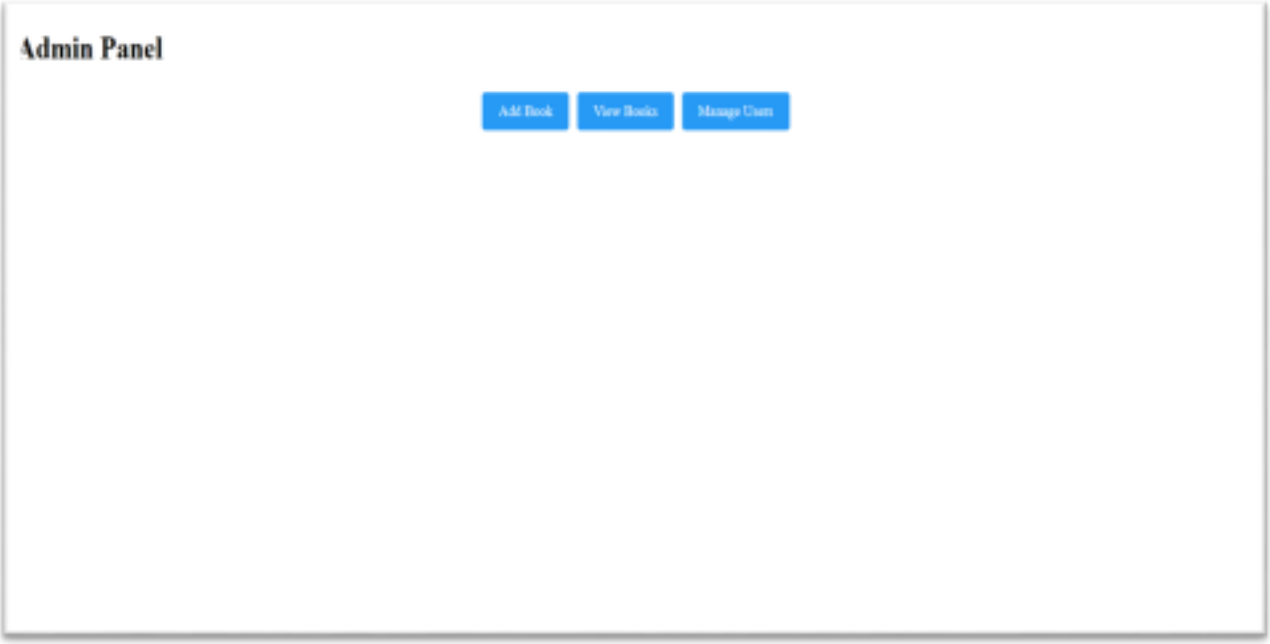
CSS Style for Admin Panel: .admin-actions {

```
display: flex;
 justify-content: center;
margin-top: 20px;
}

.admin-actions  .btn  {
 text-decoration: none;
 background-color: #2196F3;
color: white;
 padding: 10px 20px;
 border-radius:   4px;
 margin: 5px;
 text-align: center;
}

.admin-actions    .btn:hover    {
background-color: #1976D2; }
```

Output for Admin Panel:



➢ Backend Development

Database Setup (Create table for Users, Books, Transaction):

```
CREATE DATABASE library;

USE library;

CREATE TABLE Users (

 user_id  INT  AUTO_INCREMENT  PRIMARY  KEY,

name VARCHAR(100),

 email VARCHAR(100) UNIQUE,

 password      VARCHAR(100),

 role ENUM('student', 'admin')

);

CREATE TABLE Books (

 book_id  INT  AUTO_INCREMENT  PRIMARY  KEY,

title VARCHAR(255),

 author        VARCHAR(255),

 genre VARCHAR(100),

 quantity INT

);
```

```
CREATE TABLE Transactions (
 transaction_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT,
 book_id INT,
 issue_date DATE,
 return_date DATE,
 fine DECIMAL(10, 2),
 FOREIGN KEY (user_id) REFERENCES Users(user_id),
FOREIGN KEY (book_id) REFERENCES Books(book_id) );
SELECT * FROM users;
SELECT author , title FROM Books;
```

These Queries will show: User Table:

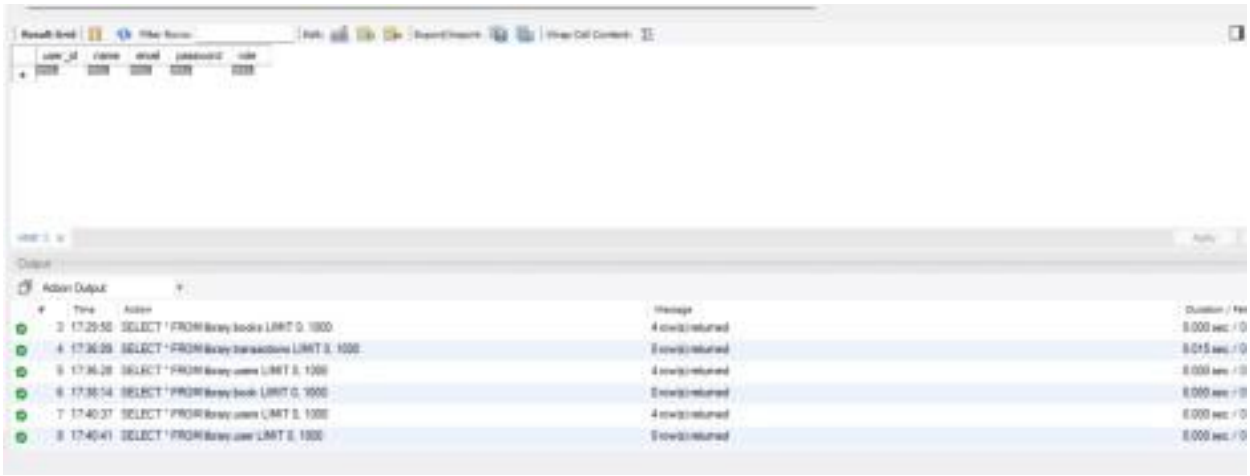


Table for books:

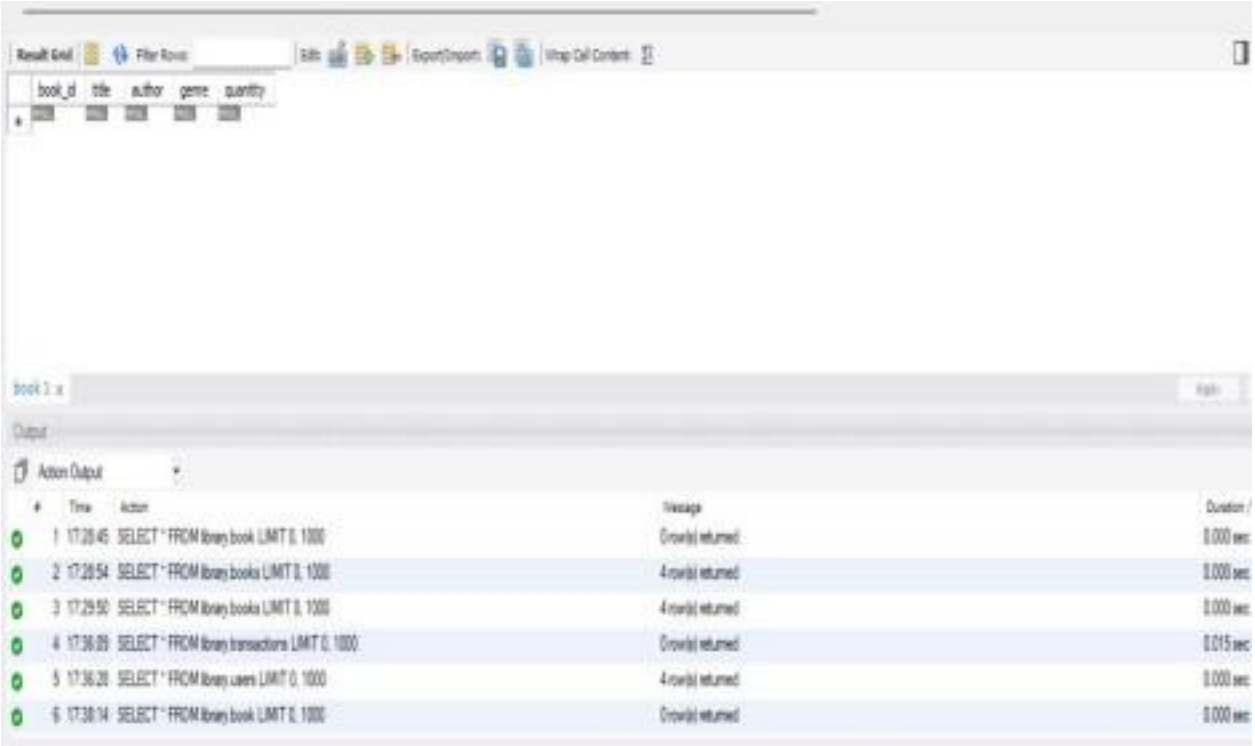
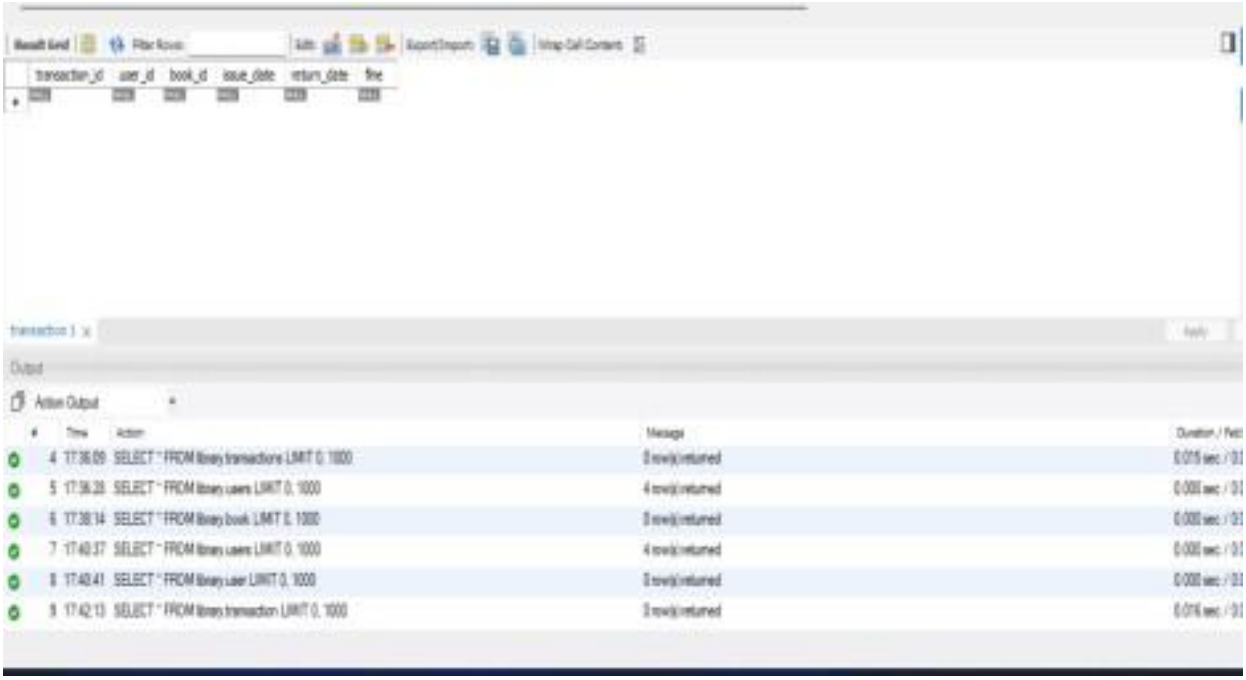


Table for Transactions:



CRUD Operations for Books (Add, Edit, Delete, View)

```
CREATE DATABASE LibraryManagement;

USE LibraryManagement;

CREATE TABLE users(

user_id int

AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(100) NOT NULL,

email VARCHAR(100)UNIQUE NOT NULL,

password VARCHAR(255)

NOT NULL,

role ENUM('student','admin')DEFAULT 'student',

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

INSERT INTO users(user_id,name,email,password)

VALUES(1,"Aarti","Aarti@gmail.com",12345678),

(2,"Radhika","Radhika@gmail.com",87654321),

(3,"Meenu","Meenu@gmail.com",43215678),
```

```
(4,"Rashi","Rashi@gmail.com",56784321);


CREATE TABLE Books (

 book_id  INT  AUTO_INCREMENT  PRIMARY  KEY,

title VARCHAR(255) NOT NULL,

 author   VARCHAR(255)   NOT   NULL,

 genre VARCHAR(50),

 quantity INT DEFAULT 1,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );

INSERT          INTO          books(book_id,title,author,genre)

VALUES(101,"machine learning","RD","Fiction"),

          (102,"1984","George          orwell","Dystopian"),

          (103,"pride   and   prejudice","jane   austen","filmy"),

          (104,"The Great gatsby","f.scott ","classic");


 SELECT * FROM Books;
```

```
CREATE TABLE Transactions (

 transaction_id INT AUTO_INCREMENT PRIMARY KEY,

user_id INT NOT NULL,

 book_id INT NOT NULL,

 transaction_type ENUM('borrow', 'return') NOT NULL,

 transaction_date TIMESTAMP DEFAULT
CURRENT_TIMESTAMP,

 FOREIGN KEY (user_id) REFERENCES Users(user_id),

FOREIGN KEY (book_id) REFERENCES Books(book_id) );


SELECT * FROM Books WHERE title LIKE'%Gatsby%';

UPDATE Books

SET quantity = 15, genre = 'Classic'

WHERE book_id = 1;

DELETE FROM Books WHERE book_id = 1;


SELECT quantity FROM Books WHERE book_id = 1;

 UPDATE Books SET quantity = quantity - 1

 WHERE book_id = 1;
```

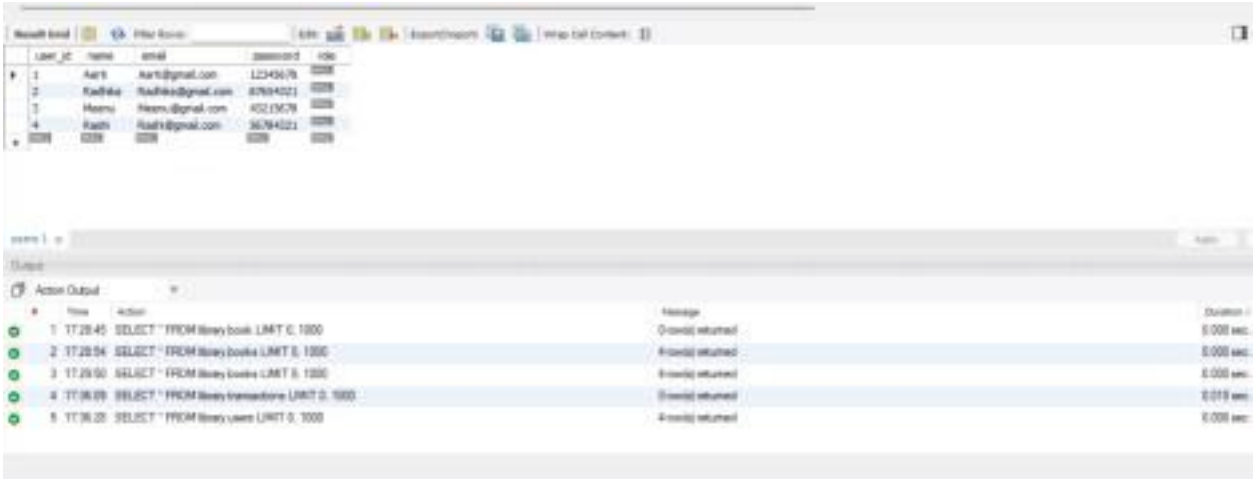
SELECT * FROM Books;

UPDATE Books SET quantity = 20 WHERE book_id = 2;

DELETE FROM Books WHERE book_id = 3;

SELECT* FROM Books;

These Queries will show:



Chapter 5

TESTING

In the Testing Phase of an online Library Management System (LMS) based on the Software Development Life Cycle (SDLC) model, the primary objective is to identify defects or bugs and ensure the system meets its requirements and functions as expected. The Testing Phase is critical for verifying and validating the system before it is deployed for production.

➢ Types of testing conducted

1. Unit Testing

- **Objective**: Test individual units or components of the system in isolation to ensure they function correctly.
- **Focus**: Testing individual functions, methods, or modules. •

Example: Testing the function that allows a user to borrow a book, ensuring that the correct book is added to the user's borrow list.

2. Integration Testing

- **Objective**: Verify that different modules or components of the system work together as intended.
- **Focus**: Interaction between integrated components (e.g., the interaction between the user registration system and the book borrowing system).
- **Example**: Ensuring that when a user borrows a book, the system updates the book's availability status and the user's borrowing history.

3. System Testing

- **Objective**: Test the entire system as a whole to ensure it meets the specified requirements and works correctly in its entirety.

- **Focus**: Testing the system’s complete functionality, ensuring all components work together seamlessly.
- **Example**: Ensuring the whole process of searching for books, borrowing, and returning them functions as expected, and the database is updated accordingly.

4. Acceptance Testing

- **Objective**: Confirm that the system meets the business and functional requirements of the stakeholders and is ready for deployment.
- **Focus**: Ensuring that the system meets user expectations and is ready for real-world use.
- **Example**: Involving users (e.g., librarians or students) to verify if they can easily search, borrow, and return books, and the system is functioning as expected.

5. Usability Testing

- **Objective**: Evaluate how user-friendly the system is and whether it is intuitive for users to interact with.
- **Focus**: User experience (UX) and interface design.
- **Example**: Ensuring the library search interface is easy to use, and users can easily navigate through options such as borrowing, returning, and searching for books.

6. Database Testing

- **Objective**: Verify that the database works as intended, ensuring data integrity, accuracy, and consistency.
- **Focus**: Testing the database's functionality, data integrity, and performance.
- **Example**: Ensuring that when a book is borrowed, the database accurately records the transaction, and that records are not duplicated or lost when users return books.

➢ Test cases and results

Testing Type	Description	Outcomes
Unit Testing	Individual modules were tested to ensure their functionality.	All units tested passed successfully with no critical errors.
Integration Testing	Integrated modules were tested to ensure they work together as expected.	Integration between modules (e.g., user management and book management) was smooth and error-free.
System Testing	The complete system was tested to validate the entire workflow.	The system performed as expected with no major issues.
Acceptance Testing	Real users tested the system to confirm it met their needs and expectations.	Users were satisfied with the functionality and ease of use.
Usability Testing	The user interface was tested to ensure it was intuitive and easy to navigate	The UI was user friendly, and users found it easy to perform tasks.

Performance Testing	The system was tested for load handling, speed, and responsiveness.	The system handled concurrent users efficiently without performance degradation.
Security Testing	The system was tested for vulnerabilities and to ensure data protection measures were in place.	The system passed security tests, with no significant vulnerabilities found.

Chapter 6

CONCLUSION

➢ Summary of the project

This system is aimed at improving the efficiency and accessibility of library operations, providing both library staff and users with an intuitive platform for managing books, memberships, borrowing, and returning items. The system allows users to search for books, borrow or return them, and manage their accounts, while providing administrators with tools for tracking inventory, managing users, and generating reports.

The development of this system follows the **Software Development Life Cycle (SDLC)** model, ensuring a structured approach to software development. The SDLC provides a systematic framework for planning, designing, implementing, testing, and maintaining the system, ensuring that the final product is robust, scalable, and meets user requirements.

The project follows the **Waterfall SDLC model**, which consists of distinct phases:

Requirement Analysis: Identifying and gathering the functional and non-functional requirements from stakeholders.

System Design: Creating the architectural design of the system, including the user interface, database design, and system flow.

Implementation (Coding): Writing the code for the application, integrating different modules, and ensuring system functionality. **Testing:** Conducting unit tests, integration tests, and user acceptance tests to ensure the system meets the defined requirements and is free from errors.

Deployment and Maintenance: Deploying the system for end-users and providing ongoing maintenance and updates based on feedback and evolving requirements.

➢ Achievements and outcomes

Improved Library Operations:

- Automated tasks like cataloging, book borrowing, and user management, reducing the need for manual work.

Better User Experience:

- Created an easy-to-use platform for users to search for books, borrow or return them, and manage their accounts online.

Efficient Admin Management:

- Provided administrators with a dashboard to track books, manage users, and generate reports easily.

Scalability and Flexibility:

- Designed the system to handle more users and books in the future, allowing for future updates.

Accurate Book Tracking:

- Implemented features like due date calculations and reminders, helping keep track of borrowed books.

Successful Testing and Launch:

- Thoroughly tested the system and deployed it successfully, ensuring it works as expected.

Positive User Feedback:

- Users and staff have given positive feedback, confirming the

system makes library operations easier and more efficient.

This section highlights the key successes and positive outcomes of the project, showing how it improved library management and user satisfaction.

The **Online Library Management System** project successfully automated library operations, improving efficiency by reducing manual tasks such as cataloging and user management. It provided an easy-to use platform for users to search for, borrow, and return books, while administrators gained effective tools for managing books, users, and generating reports. The system was designed to be scalable to accommodate future growth. Key features like accurate book tracking and due date reminders enhanced overall management. After thorough testing, the system was successfully deployed, receiving positive feedback from both users and administrators.

➢ Future improvements or extensions

Mobile App:

- Develop a mobile version for easier access to library services on smartphones.

E-books and Audiobooks:

- Add support for borrowing e-books and audiobooks along with physical books.

Better Search Options:

- Improve search features with filters like genre, author, and user ratings to help users find books more easily.

Automated Alerts:

- Send automatic notifications for overdue books, reservations, and reminders via email or SMS.

Online Payments:

- Allow users to pay fines, fees, or book reservations online for added convenience.

Multi-language Support:

- Offer the system in multiple languages to make it accessible to more users.

Book Recommendations:

- Use AI to recommend books based on users' reading history.

Cloud Storage:

- Move the system to the cloud for better security and easy access from different devices.

These improvements will make the system more user-friendly and expand its features.

REFERENCES

- W3Schools
- Wikipedia
- CSS Tricks
- MySQL Workbench